

Document Number: P0244R0

Date: 2016-02-10

Audience: Library Evolution Working Group

Reply-to: Tom Honermann <tom@honermann.net>

Text_view: A C++ concepts and range based character encoding and code point enumeration library

- [Introduction](#)
- [Motivation and Scope](#)
- [Terminology](#)
 - [Code Unit](#)
 - [Code Point](#)
 - [Character Set](#)
 - [Character](#)
 - [Encoding](#)
- [Design Considerations](#)
 - [Iterator Requirement Conformance](#)
 - [Error Handling](#)
 - [Encoding Forms vs Encoding Schemes](#)
 - [Streaming](#)
- [Implementation Experience](#)
- [Technical Specifications](#)
 - [Header <text_view> synopsis](#)
 - [Concepts](#)
 - [Character Sets](#)
 - [Character Set Identification](#)
 - [Character Set Information](#)
 - [Characters](#)
 - [Encodings](#)
 - [Text Iterators](#)
 - [Text View](#)
- [Acknowledgements](#)
- [References](#)

Introduction

C++11 [\[C++11\]](#) added support for new character types [\[N2249\]](#) and Unicode string literals [\[N2442\]](#), but neither C++11, nor more recent standards have provided means of efficiently and conveniently enumerating code points in Unicode or legacy encodings. While it is possible to implement such enumeration using interfaces provided in the standard `<locale>` and `<codecvt>` libraries, doing so is awkward, requires that text be provided as pointers to contiguous memory, and inefficient due to virtual function call overhead.

The described library provides iterator and range based interfaces for encoding and decoding strings in a variety of character encodings. The interface is intended to support all modern and legacy character encodings, though implementations are expected to only provide support for a limited set of encodings.

An example usage follows. Note that `\u00F8` (LATIN SMALL LETTER O WITH STROKE) is encoded as UTF-8 using two code units (`\xC3\xB8`), but iterator based enumeration sees just the single code point.

```
using CT = utf8_encoding::character_type;
auto tv = make_text_view(u8"J\u00F8erg");
auto it = tv.begin();
assert(*it++ == CT{0x004A}); // 'J'
assert(*it++ == CT{0x00F8}); // 'ø'
assert(*it++ == CT{0x0065}); // 'e'
```

The provided iterators and views are compatible with the non-modifying sequence utilities provided by the standard C++ `<algorithm>` library. This enables use of standard algorithms to search encoded text.

```
it = std::find(tv.begin(), tv.end(), CT{0x00F8});
assert(it != tv.end());
```

The iterators also provide access to the underlying code unit sequence.

```
auto base_it = it.base_range().begin();
assert(*base_it++ == '\xC3');
assert(*base_it++ == '\xB8');
assert(base_it == it.base_range().end());
```

These ranges satisfy the requirements for use in C++11 range-based for statements. This support is currently limited to views constructed for stateless encodings as a sentinel type is used as the end iterator for stateful encodings. The enhancements to the range-based for statement in the ranges proposal [\[Ranges\]](#) will remove this limitation.

```
for (const auto& ch : tv) {
    ...
}
```

Motivation and Scope

Consider the following code to search for the occurrence of U+00F8 in the UTF-8 encoded string using C++ standard provided interfaces.

```
std::string s = u8"J\u00F8erg";
std::mbstate_t state = std::mbstate_t{};
codecvt_utf8<char32_t> utf8_converter;
const char *from_begin = s.data();
const char *from_end = s.data() + s.size();
const char *from_current;
const char *from_next = from_begin;
char32_t to[1];
std::codecvt_base::result r;
do {
    from_current = from_next;
    char32_t *to_begin = &to[0];
    char32_t *to_end = &to[1];
    char32_t *to_next;
    r = utf8_converter.in(
        state,
        from_current, from_end, from_next,
        to_begin, to_end, to_next);
} while (r != std::codecvt_base::error && to[0] != char32_t{0x00F8});
if (r != std::codecvt_base::error && to[0] == char32_t{0x00F8}) {
    cout << "Found at offset " << (from_current - from_begin) << endl;
} else {
    cout << "Not found" << endl;
}
```

There are a number of issues with the above code:

- It is verbose.
- It is limited to working with strings that are stored in contiguous memory.
- It is inefficient. All `codecvt` public member functions dispatch to virtual member functions.
- It is not generic. Use of the `codecvt_utf8` facet makes it specific to handling of UTF-8 encoded text. Making this code generic would require some other means of identifying an appropriate facet to use.
- It is not applicable to non-Unicode encodings; `codecvt` doesn't provide means to retrieve a code point for the encodings used for ordinary and wide strings. The above code only accomplishes this by depending on transcoding to UTF-32 and the fact that UTF-32 is a trivial encoding.

The above method is not the only method available to identify a search term in an encoded string. For some encodings, it is feasible to encode the search term in the encoding and to search for a matching code unit sequence. This approach works for UTF-8, UTF-16, and UTF-32, but not for many other encodings. Consider the Shift-JIS encoding of U+6D6C. This is encoded as 0x8A 0x5C. Shift-JIS is a multibyte encoding that is almost ASCII compatible. The code unit sequence 0x5C encodes the ASCII `'\'` character. But note that 0x5C appears as the second byte of the code unit sequence for U+6D6C. Naively searching for the matching code unit sequence for `'\'` would incorrectly match the trailing code unit sequence for U+6D6C.

The library described here is intended to solve the above issues while also providing a modern interface that is intuitive to use and can be used with other standard provided facilities; in particular, the C++ standard `<algorithm>` library.

Terminology

The terminology used in this document is intended to be consistent with industry standards and, in particular, the Unicode standard. Any inconsistencies in the use of this terminology and that in the Unicode standard is unintentional. The terms described in this document comprise a subset of the terminology used within the Unicode standard; only those terms necessary to specify functionality exhibited by the proposed library are included here. Those who would like to learn more about general text processing terminology in computer systems are encouraged to read chapter 2, "General Structure" of the Unicode standard.

Code Unit

A single, indivisible, integral element of an encoded sequence of characters. A sequence of one or more code units specifies a code point or encoding state transition as defined by a character encoding. A code unit does not, by itself, identify any particular character or code point; the meaning ascribed to a particular code unit value is derived from a character encoding definition.

The `char`, `wchar_t`, `char16_t`, and `char32_t` types are most commonly used as code unit types.

The string literal `u8"J\u00F8erg"` contains 7 code units and 6 code unit sequences; `"\u00F8"` is encoded in UTF-8 using two code units and string literals contain a trailing NUL code unit.

The string literal `"J\u00F8erg"` contains an implementation defined number of code units. The standard does not specify the encoding of ordinary and wide string literals, so the number of code units encoded by `"\u00F8"` depends on the implementation defined encoding used for ordinary string literals.

Code Point

An integral value denoting an abstract character as defined by a character set. A code point does not, by itself, identify any particular character; the meaning ascribed to a particular code point value is derived from a character set definition.

The `char`, `wchar_t`, `char16_t`, and `char32_t` types are most commonly used as code point types.

The string literal `u8"J\u00F8erg"` describes a sequence of 6 code point values; string literals implicitly specify a trailing NUL code point.

The string literal `"J\u00F8erg"` describes a sequence of an implementation defined number of code point values. The standard does not specify the encoding of ordinary and wide string literals, so the number of code points encoded by `"\u00F8"` depends on the implementation defined encoding used for ordinary string literals. Implementations are permitted to translate a single code point in the source or Unicode character sets to multiple code points in the execution encoding.

Character Set

A mapping of code point values to abstract characters. A character set need not provide a mapping for every possible code point value representable by the code point type.

C++ does not specify the use of any particular character set or encoding for ordinary and wide character and string literals, though it does place some restrictions on them. Unicode character and string literals are governed by the Unicode standard.

Common character sets include ASCII, Unicode, and Windows code page 1252.

Character

An element of written language, for example, a letter, number, or symbol. A character is identified by the combination of a character set and a code point value.

Encoding

A method of representing a sequence of characters as a sequence of code unit sequences.

An encoding may be stateless or stateful. In stateless encodings, characters may be encoded or decoded starting from the beginning of any code unit sequence. In stateful encodings, it may be necessary to record certain affects of previously encoded characters in order to correctly encode additional characters, or to decode preceding code unit sequences in order to correctly decode following code unit sequences.

An encoding may be fixed width or variable width. In fixed width encodings, all characters are encoded using a single code unit

sequence and all code unit sequences have the same length. In variable width encodings, different characters may require multiple code unit sequences, or code unit sequences of varying length.

An encoding may support bidirectional or random access decoding of code unit sequences. In bidirectional encodings, characters may be decoded by traversing code unit sequences in reverse order. Such encodings must support a method to identify the start of a preceding code unit sequence. In random access encodings, characters may be decoded from any code unit sequence within the sequence of code unit sequences, in constant time, without having to decode any other code unit sequence. Random access encodings are necessarily stateless and fixed length. An encoding that is neither bidirectional, nor random access, may only be decoded by traversing code unit sequences in forward order.

An encoding may support encoding characters from multiple character sets. Such an encoding is either stateful and defines code unit sequences that switch the active character set, or defines code unit sequences that implicitly identify a character set, or both.

A trivial encoding is one in which all encoded characters correspond to a single character set and where each code unit encodes exactly one character using the same value as the code point for that character. Such an encoding is stateless, fixed width, and supports random access decoding.

Common encodings include the Unicode UTF-8, UTF-16, and UTF-32 encodings, the ISO/IEC 8859 series of encodings including ISO/IEC 8859-1, and many trivial encodings such as Windows code page 1252.

Design Considerations

Iterator Requirement Conformance

The iterators provided by this library do not conform to all of the C++ standard requirements for forward and random access iterators.

Each iterator holds its own copy of decoded code point values, two iterators that compare equally will return different addresses when dereferenced. The standard requires that equivalent iterators return equivalent reference and pointer addresses when dereferenced. For random access iterators, `operator[]` returns a value type since any returned reference type would immediately become dangling.

The above conformance issues will be resolved if the proxy iterators proposal [P0022R1](#) is accepted.

Error Handling

The reference implementation currently throws exceptions when underflow occurs or when invalid code unit sequences are encountered. Use of exceptions is not acceptable by many members of the C++ community.

An alternative to exceptions has not yet been settled on. One possibility is to add an additional template parameter to the `basic_text_view` and `itext_iterator` class templates that enables alternative error handling to be implemented. Custom error handlers could then substitute replacement characters and/or record errors via some other mechanism.

Encoding Forms vs Encoding Schemes

The Unicode standard differentiates code unit oriented and byte oriented encodings. The former are termed encoding forms; the latter, encoding schemes. This library provides support for some of each. For example, `utf16_encoding` is code unit oriented; the value type of its iterators is `char16_t`. The `utf16be_encoding`, `utf16le_encoding`, and `utf16bom_encoding` encodings are byte oriented; the value type of their iterators is `char`.

Streaming

Decoding from a streaming source without unacceptably blocking on underflow requires the ability to decode a partial code unit sequence, save state, and then resume decoding the remainder of the code unit sequence when more data becomes available. This requirement presents challenges for an iterator based approach. The specification presented in this paper does not provide a good solution for this use case.

One possibility is to add additional state tracking that is stored with each iterator. Support for the possibility of trailing non-code-point encoding code unit sequences (escape sequences in some encodings) already requires that code point iterators greedily consume code units. This enables an iterator to compare equal to the end iterator even when its current base code unit iterator does not equal the end iterator of the underlying code unit range. Storing partial code unit sequence state with an iterator that

compares equal to the end iterator would enable users to write code like the following.

```
using encoding = utf8_encoding;
auto state = encoding::initial_state();
do {
    std::string b = get_more_data();
    auto tv = make_text_view<encoding>(state, begin(b), end(b));
    auto it = begin(tv);
    while (it != end(tv))
        ...;
    state = it; // Trailing state is preserved in the end iterator. Save it
               // to seed state for the next loop iteration.
} while (!b.empty());
```

However, this leaves open the possibility for trailing code units at the end of an encoded text to go unnoticed. In a non-buffering scenario, an iterator might silently compare equal to the end iterator even though there are (possibly invalid) code units remaining.

It might be feasible to address this by adding a policy template parameter to `basic_text_view` and `itext_iterator` similar to what is discussed in the [error handling](#) section.

Implementation Experience

A reference implementation of the described library is publicly available at https://github.com/tahonermann/text_view [Text view]. The implementation requires a compiler that implements the C++ Concepts technical specification [Concepts]. The only compiler known to do so at the time of this writing is the in-development gcc 6.0 release.

The reference implementation currently depends on Andrew Sutton's [Origin](#) [Origin] libraries for concept definitions. Origin's concept definitions do not match the concept definitions specified in the proposed ranges technical specification [Ranges] and used as the specification of the described library in this document. As a result, the interface declarations in the reference implementation differ from those presented here. The expectation is that code written to the specification presented here will work with the reference implementation, but there may be some corner cases that make the differences apparent. Any such differences should be considered defects or limitations of the reference implementation and reported at https://github.com/tahonermann/text_view/issues.

Technical Specifications

Header `<text_view>` synopsis

```
namespace std {
namespace experimental {
inline namespace text {

// concepts:
template<typename T> concept bool CodeUnit();
template<typename T> concept bool CodePoint();
template<typename T> concept bool CharacterSet();
template<typename T> concept bool Character();
template<typename T> concept bool CodeUnitIterator();
template<typename T, typename V> concept bool CodeUnitOutputIterator();
template<typename T> concept bool TextEncodingState();
template<typename T> concept bool TextEncodingStateTransition();
template<typename T> concept bool TextEncoding();
template<typename T, typename I> concept bool TextEncoder();
template<typename T, typename I> concept bool TextDecoder();
template<typename T, typename I> concept bool TextForwardDecoder();
template<typename T, typename I> concept bool TextBidirectionalDecoder();
template<typename T, typename I> concept bool TextRandomAccessDecoder();
template<typename T> concept bool TextIterator();
template<typename T> concept bool TextOutputIterator();
template<typename T, typename I> concept bool TextSentinel();
template<typename T> concept bool TextView();

// character sets:
class any_character_set;
class basic_execution_character_set;
class basic_execution_wide_character_set;
class unicode_character_set;
```

```

// implementation defined character set type aliases:
using execution_character_set = /* implementation-defined */ ;
using execution_wide_character_set = /* implementation-defined */ ;
using universal_character_set = /* implementation-defined */ ;

// character set identification:
class character_set_id;

template<typename CST>
    inline character_set_id get_character_set_id();

// character set information:
class character_set_info;

template<typename CST>
    inline const character_set_info& get_character_set_info();
const character_set_info& get_character_set_info(character_set_id id);

// character set and encoding traits:
template<typename T>
    using code_unit_type_t = /* implementation-defined */ ;
template<typename T>
    using code_point_type_t = /* implementation-defined */ ;
template<typename T>
    using character_set_type_t = /* implementation-defined */ ;
template<typename T>
    using character_type_t = /* implementation-defined */ ;
template<typename T>
    using encoding_type_t /* implementation-defined */ ;

// characters:
template<CharacterSet CST> class character;
template <> class character<any_character_set>;

template<CharacterSet CST>
    bool operator==(const character<any_character_set> &lhs,
                    const character<CST> &rhs);
template<CharacterSet CST>
    bool operator==(const character<CST> &lhs,
                    const character<any_character_set> &rhs);
template<CharacterSet CST>
    bool operator!=(const character<any_character_set> &lhs,
                    const character<CST> &rhs);
template<CharacterSet CST>
    bool operator!=(const character<CST> &lhs,
                    const character<any_character_set> &rhs);

// encoding state and transition types:
class trivial_encoding_state;
class trivial_encoding_state_transition;
class utf8bom_encoding_state;
class utf8bom_encoding_state_transition;
class utf16bom_encoding_state;
class utf16bom_encoding_state_transition;
class utf32bom_encoding_state;
class utf32bom_encoding_state_transition;

// encodings:
class basic_execution_character_encoding;
class basic_execution_wide_character_encoding;
#if defined(__STDC_ISO_10646__)
class iso_10646_wide_character_encoding;
#endif // __STDC_ISO_10646__
class utf8_encoding;
class utf8bom_encoding;
class utf16_encoding;
class utf16be_encoding;
class utf16le_encoding;
class utf16bom_encoding;
class utf32_encoding;
class utf32be_encoding;
class utf32le_encoding;
class utf32bom_encoding;

// implementation defined encoding type aliases:
using execution_character_encoding = /* implementation-defined */ ;
using execution_wide_character_encoding = /* implementation-defined */ ;
using char8_character_encoding = /* implementation-defined */ ;

```

```

using char16_character_encoding = /* implementation-defined */ ;
using char32_character_encoding = /* implementation-defined */ ;

// itext_iterator:
template<TextEncoding ET, ranges::InputRange RT>
    requires TextDecoder<ET, ranges::iterator_t<const RT>>()
    class itext_iterator;

// itext_sentinel:
template<TextEncoding ET, ranges::InputRange RT>
    class itext_sentinel;

// otext_iterator:
template<TextEncoding E, CodeUnitOutputIterator<code_unit_type_t<E>> CUIT>
    class otext_iterator;

// otext_iterator factory functions:
template<TextEncoding ET, CodeUnitOutputIterator<code_unit_type_t<ET>> IT>
    auto make_otext_iterator(typename ET::state_type state, IT out)
        -> otext_iterator<ET, IT>;
template<TextEncoding ET, CodeUnitOutputIterator<code_unit_type_t<ET>> IT>
    auto make_otext_iterator(IT out)
        -> otext_iterator<ET, IT>;

// basic_text_view:
template<TextEncoding ET, ranges::InputRange RT>
    class basic_text_view;

// basic_text_view type aliases:
using text_view = basic_text_view<execution_character_encoding,
                                /* implementation-defined */ >;
using wtext_view = basic_text_view<execution_wide_character_encoding,
                                /* implementation-defined */ >;
using u8text_view = basic_text_view<char8_character_encoding,
                                /* implementation-defined */ >;
using u16text_view = basic_text_view<char16_character_encoding,
                                /* implementation-defined */ >;
using u32text_view = basic_text_view<char32_character_encoding,
                                /* implementation-defined */ >;

// basic_text_view factory functions:
template<TextEncoding ET, ranges::InputIterator IT, ranges::Sentinel<IT> ST>
    auto make_text_view(typename ET::state_type state, IT first, ST last)
        -> basic_text_view<ET, /* implementation-defined */ >;
template<TextEncoding ET, ranges::InputIterator IT, ranges::Sentinel<IT> ST>
    auto make_text_view(IT first, ST last)
        -> basic_text_view<ET, /* implementation-defined */ >;
template<TextEncoding ET, ranges::ForwardIterator IT>
    auto make_text_view(typename ET::state_type state,
                        IT first,
                        typename std::make_unsigned<ranges::difference_type_t<IT>>::type n)
        -> basic_text_view<ET, /* implementation-defined */ >;
template<TextEncoding ET, ranges::ForwardIterator IT>
    auto make_text_view(IT first,
                        typename std::make_unsigned<ranges::difference_type_t<IT>>::type n)
        -> basic_text_view<ET, /* implementation-defined */ >;
template<TextEncoding ET, ranges::InputRange Iterable>
    auto make_text_view(typename ET::state_type state,
                        const Iterable &iterable)
        -> basic_text_view<ET, /* implementation-defined */ >;
template<TextEncoding ET, ranges::InputRange Iterable>
    auto make_text_view(const Iterable &iterable)
        -> basic_text_view<ET, /* implementation-defined */ >;
template<TextIterator TIT, TextSentinel<TIT> TST>
    auto make_text_view(TIT first, TST last)
        -> basic_text_view<ET, /* implementation-defined */ >;
template<TextView TVT>
    TVT make_text_view(TVT tv);

} // inline namespace text
} // namespace experimental
} // namespace std

```

Concepts

- [Concept CodeUnit](#)
- [Concept CodePoint](#)

- [Concept CharacterSet](#)
- [Concept Character](#)
- [Concept CodeUnitIterator](#)
- [Concept CodeUnitOutputIterator](#)
- [Concept TextEncodingState](#)
- [Concept TextEncodingStateTransition](#)
- [Concept TextEncoding](#)
- [Concept TextEncoder](#)
- [Concept TextDecoder](#)
- [Concept TextForwardDecoder](#)
- [Concept TextBidirectionalDecoder](#)
- [Concept TextRandomAccessDecoder](#)
- [Concept TextIterator](#)
- [Concept TextSentinel](#)
- [Concept TextOutputIterator](#)
- [Concept TextView](#)

Concept CodeUnit

The CodeUnit concept specifies requirements for a type usable as the code unit type of a string type.

CodeUnit<T>() is satisfied if and only if:

- `std::is_integral<T>::value` is true
- and at least one of:
 - `std::is_unsigned<T>::value` is true.
 - `std::is_same<std::remove_cv<T>::type, char>::value` is true.
 - `std::is_same<std::remove_cv<T>::type, wchar_t>::value` is true.

```
template<typename T> concept bool CodeUnit() {
    return /* implementation-defined */ ;
}
```

Concept CodePoint

The CodePoint concept specifies requirements for a type usable as the code point type of a character set type.

CodePoint<T>() is satisfied if and only if:

- `std::is_integral<T>::value` is true
- and at least one of:
 - `std::is_unsigned<T>::value` is true.
 - `std::is_same<std::remove_cv<T>::type, char>::value` is true.
 - `std::is_same<std::remove_cv<T>::type, wchar_t>::value` is true.

```
template<typename T> concept bool CodePoint() {
    return /* implementation-defined */ ;
}
```

Concept CharacterSet

The CharacterSet concept specifies requirements for a type that describes a character set. Such a type has a member typedef-name declaration for a type that satisfies CodePoint and a static member function that returns a name for the character set.

```
template<typename T> concept bool CharacterSet() {
    return CodePoint<code_point_type_t<T>>()
        && requires () {
            { T::get_name() } noexcept -> const char *;
        };
}
```

Concept Character

The Character concept specifies requirements for a type that describes a character as defined by an associated character set. Non-static member functions provide access to the code point value of the described character. Types that satisfy Character are

regular and copyable.

```
template<typename T> concept bool Character() {
    return ranges::Regular<T>()
        && ranges::Copyable<T>()
        && CharacterSet<character_set_type_t<T>>()
        && requires (T t, code_point_type_t<character_set_type_t<T>> cp) {
            t.set_code_point(cp);
            { t.get_code_point() } -> code_point_type_t<character_set_type_t<T>>;
            { t.get_character_set_id() } -> character_set_id;
        };
}
```

Concept CodeUnitIterator

The CodeUnitIterator concept specifies requirements of an iterator that has a value type that satisfies CodeUnit.

```
template<typename T> concept bool CodeUnitIterator() {
    return ranges::Iterator<T>()
        && CodeUnit<ranges::value_type_t<T>>();
}
```

Concept CodeUnitOutputIterator

The CodeUnitOutputIterator concept specifies requirements of an output iterator that can be assigned from a type that satisfies CodeUnit.

```
template<typename T, typename V> concept bool CodeUnitOutputIterator() {
    return ranges::OutputIterator<T, V>()
        && CodeUnit<V>();
}
```

Concept TextEncodingState

The TextEncodingState concept specifies requirements of types that hold encoding state. Such types are default constructible and copyable.

```
template<typename T> concept bool TextEncodingState() {
    return ranges::DefaultConstructible<T>()
        && ranges::Copyable<T>();
}
```

Concept TextEncodingStateTransition

The TextEncodingStateTransition concept specifies requirements of types that hold encoding state transitions. Such types are default constructible and copyable.

```
template<typename T> concept bool TextEncodingStateTransition() {
    return ranges::DefaultConstructible<T>()
        && ranges::Copyable<T>();
}
```

Concept TextEncoding

The TextEncoding concept specifies requirements of types that define an encoding. Such types define member types that identify the code unit, character, encoding state, and encoding state transition types, a static member function that returns an initial encoding state object that defines the encoding state at the beginning of a sequence of encoded characters, and static data members that specify the minimum and maximum number of code units used to encode any single character.

```
template<typename T> concept bool TextEncoding() {
    return requires () {
        { T::min_code_units } noexcept -> int;
        { T::max_code_units } noexcept -> int;
    }
    && TextEncodingState<typename T::state_type>()
    && TextEncodingStateTransition<typename T::state_transition_type>()
    && CodeUnit<code_unit_type_t<T>>()
}
```

```

    && Character<character_type_t<T>>>()
    && requires () {
        { T::initial_state() }
        -> const typename T::state_type&;
    };
}

```

Concept TextEncoder

The `TextEncoder` concept specifies requirements of types that are used to encode characters using a particular code unit iterator that satisfies `OutputIterator`. Such a type satisfies `TextEncoding` and defines static member functions used to encode state transitions and characters.

```

template<typename T, typename I> concept bool TextEncoder() {
    return TextEncoding<T>()
        && ranges::OutputIterator<CUIT, code_unit_type_t<T>>>()
        && requires (
            typename T::state_type &state,
            CUIT &out,
            typename T::state_transition_type stt,
            int &encoded_code_units)
        {
            T::encode_state_transition(state, out, stt, encoded_code_units);
        }
        && requires (
            typename T::state_type &state,
            CUIT &out,
            character_type_t<T> c,
            int &encoded_code_units)
        {
            T::encode(state, out, c, encoded_code_units);
        }
    };
}

```

Concept TextDecoder

The `TextDecoder` concept specifies requirements of types that are used to decode characters using a particular code unit iterator that satisfies `InputIterator`. Such a type satisfies `TextEncoding` and defines a static member function used to decode state transitions and characters.

```

template<typename T, typename I> concept bool TextDecoder() {
    return TextEncoding<T>()
        && ranges::InputIterator<CUIT>()
        && ranges::ConvertibleTo<ranges::value_type_t<CUIT>,
            code_unit_type_t<T>>>()
        && requires (
            typename T::state_type &state,
            CUIT &in_next,
            CUIT in_end,
            character_type_t<T> &c,
            int &decoded_code_units)
        {
            { T::decode(state, in_next, in_end, c, decoded_code_units) } -> bool;
        }
    };
}

```

Concept TextForwardDecoder

The `TextForwardDecoder` concept specifies requirements of types that are used to decode characters using a particular code unit iterator that satisfies `ForwardIterator`. Such a type also satisfies `TextDecoder`.

```

template<typename T, typename I> concept bool TextForwardDecoder() {
    return TextDecoder<T, CUIT>()
        && ranges::ForwardIterator<CUIT>();
}

```

Concept TextBidirectionalDecoder

The `TextBidirectionalDecoder` concept specifies requirements of types that are used to decode characters using a particular code unit iterator that satisfies `BidirectionalIterator`. Such a type also satisfies `TextForwardDecoder` and defines a static member function used to decode state transitions and characters in the reverse order of their encoding.

```
template<typename T, typename I> concept bool TextBidirectionalDecoder() {
    return TextForwardDecoder<T, CUIT>()
        && ranges::BidirectionalIterator<CUIT>()
        && requires (
            typename T::state_type &state,
            CUIT &in_next,
            CUIT in_end,
            character_type_t<T> &c,
            int &decoded_code_units)
        {
            { T::rdecode(state, in_next, in_end, c, decoded_code_units) } -> bool;
        };
}
```

Concept TextRandomAccessDecoder

The `TextRandomAccessDecoder` concept specifies requirements of types that are used to decode characters using a particular code unit iterator that satisfies `RandomAccessIterator`. Such a type also satisfies `TextBidirectionalDecoder`, requires that the minimum and maximum number of code units used to encode any character have the same value, and that the encoding state be an empty type.

```
template<typename T, typename I> concept bool TextRandomAccessDecoder() {
    return TextBidirectionalDecoder<T, CUIT>()
        && ranges::RandomAccessIterator<CUIT>()
        && T::min_code_units == T::max_code_units
        && std::is_empty<typename T::state_type>::value;
}
```

Concept TextIterator

The `TextIterator` concept specifies requirements of types that are used to iterator over characters in an encoded sequence of code units. Encoding state is held in each iterator instance as needed to decode the code unit sequence and is made accessible via non-static member functions. The value type of a `TextIterator` satisfies `Character`.

```
template<typename T> concept bool TextIterator() {
    return ranges::Iterator<T>()
        && Character<ranges::value_type_t<T>>()
        && TextEncoding<encoding_type_t<T>>()
        && TextEncodingState<typename T::state_type>()
        && requires (T t, const T ct) {
            { t.state() } noexcept
                -> typename encoding_type_t<T>::state_type&;
            { ct.state() } noexcept
                -> const typename encoding_type_t<T>::state_type&;
        };
}
```

Concept TextSentinel

The `TextSentinel` concept specifies requirements of types that are used to mark the end of a range of encoded characters. A type `T` that satisfies `TextIterator` also satisfies `TextSentinel<T>` there by enabling `TextIterator` types to be used as sentinels.

```
template<typename T, typename I> concept bool TextSentinel() {
    return ranges::Sentinel<T, I>()
        && TextIterator<I>();
}
```

Concept TextOutputIterator

The `TextOutputIterator` concept specifies requirements of types that are used to encode characters as a sequence of code units. Encoding state is held in each iterator instance as needed to encode the code unit sequence and is made accessible via non-static member functions.

```
template<typename T> concept bool TextOutputIterator() {
    return ranges::OutputIterator<T, character_type_t<encoding_type_t<T>>>()
        && TextEncoding<encoding_type_t<T>>()
        && TextEncodingState<typename T::state_type>()
        && requires (T t, const T ct) {
            { t.state() } noexcept
        };
}
```

```

        -> typename encoding_type_t<T>::state_type&;
    { ct.state() } noexcept
        -> const typename encoding_type_t<T>::state_type&;
    };
}

```

Concept TextView

The `TextView` concept specifies requirements of types that provide view access to an underlying code unit range. Such types satisfy `ranges::View`, provide iterators that satisfy `TextIterator`, define member types that identify the encoding, encoding state, and underlying code unit range and iterator types. Non-static member functions are provided to access the underlying code unit range and initial encoding state.

Types that satisfy `TextView` do not own the underlying code unit range and are copyable in constant time. The lifetime of the underlying range must exceed the lifetime of referencing `TextView` objects.

```

template<typename T> concept bool TextView() {
    return ranges::View<T>()
        && TextIterator<ranges::iterator_t<T>>()
        && TextEncoding<encoding_type_t<T>>()
        && ranges::InputRange<typename T::range_type>()
        && TextEncodingState<typename T::state_type>()
        && CodeUnitIterator<code_unit_iterator_t<T>>()
        && requires (T t, const T ct) {
            { t.base() } noexcept
                -> typename T::range_type&;
            { ct.base() } noexcept
                -> const typename T::range_type&;
            { t.initial_state() } noexcept
                -> typename T::state_type&;
            { ct.initial_state() } noexcept
                -> const typename T::state_type&;
        };
}

```

Character Sets

- [Class any character set](#)
- [Class basic execution character set](#)
- [Class basic execution wide character set](#)
- [Class unicode character set](#)
- [Character set type aliases](#)

Class any_character_set

```

class any_character_set {
public:
    using code_point_type = /* implementation-defined */;

    static const char* get_name() noexcept;
};

```

Class basic_execution_character_set

```

class basic_execution_character_set {
public:
    using code_point_type = char;

    static const char* get_name() noexcept;
};

```

Class basic_execution_wide_character_set

```

class basic_execution_wide_character_set {
public:
    using code_point_type = wchar_t;

    static const char* get_name() noexcept;
};

```

```
};
```

Class unicode_character_set

```
class unicode_character_set {
public:
    using code_point_type = char32_t;

    static const char* get_name() noexcept;
};
```

Character set type aliases

```
using execution_character_set = /* implementation-defined */ ;
using execution_wide_character_set = /* implementation-defined */ ;
using universal_character_set = /* implementation-defined */ ;
```

Character Set Identification

- [Class character_set_id](#)
- [get_character_set_id](#)

Class character_set_id

```
class character_set_id {
public:
    character_set_id() = delete;

    friend bool operator==(character_set_id lhs, character_set_id rhs);
    friend bool operator!=(character_set_id lhs, character_set_id rhs);

    friend bool operator<(character_set_id lhs, character_set_id rhs);
    friend bool operator>(character_set_id lhs, character_set_id rhs);
    friend bool operator<=(character_set_id lhs, character_set_id rhs);
    friend bool operator>=(character_set_id lhs, character_set_id rhs);
};
```

get_character_set_id

```
template<typename CST>
    inline character_set_id get_character_set_id();
```

Character Set Information

- [Class character_set_info](#)
- [get_character_set_info](#)

Class character_set_info

```
class character_set_info {
public:
    character_set_info() = delete;

    character_set_id get_id() const noexcept;

    const char* get_name() const noexcept;

private:
    character_set_id id; // exposition only
};
```

get_character_set_info

```
const character_set_info& get_character_set_info(character_set_id id);
```

```
template<typename CST>
```

```
inline const character_set_info& get_character_set_info();
```

Characters

- [Class template character](#)

Class template character

```
template<CharacterSet CST>
class character {
public:
    using character_set_type = CST;
    using code_point_type = code_point_type_t<character_set_type>;

    character() = default;
    explicit character(code_point_type code_point);

    friend bool operator==(const character &lhs, const character &rhs);
    friend bool operator!=(const character &lhs, const character &rhs);

    void set_code_point(code_point_type code_point);
    code_point_type get_code_point() const;

    static character_set_id get_character_set_id();

private:
    code_point_type code_point; // exposition only
};

template<>
class character<any_character_set> {
public:
    using character_set_type = any_character_set;
    using code_point_type = code_point_type_t<character_set_type>;

    character() = default;
    explicit character(code_point_type code_point);
    character(character_set_id cs_id, code_point_type code_point);

    friend bool operator==(const character &lhs, const character &rhs);
    friend bool operator!=(const character &lhs, const character &rhs);

    void set_code_point(code_point_type code_point);
    code_point_type get_code_point() const;

    void set_character_set_id(character_set_id new_cs_id);
    character_set_id get_character_set_id() const;

private:
    character_set_id cs_id; // exposition only
    code_point_type code_point; // exposition only
};

template<CharacterSet CST>
    bool operator==(const character<any_character_set> &lhs,
                    const character<CST> &rhs);
template<CharacterSet CST>
    bool operator==(const character<CST> &lhs,
                    const character<any_character_set> &rhs);
template<CharacterSet CST>
    bool operator!=(const character<any_character_set> &lhs,
                    const character<CST> &rhs);
template<CharacterSet CST>
    bool operator!=(const character<CST> &lhs,
                    const character<any_character_set> &rhs);
```

Encodings

- [class trivial encoding state](#)
- [class trivial encoding state transition](#)
- [Class basic execution character encoding](#)
- [Class basic execution wide character encoding](#)
- [Class iso 10646 wide character encoding](#)

- [Class utf8_encoding](#)
- [Class utf8bom_encoding](#)
- [Class utf16_encoding](#)
- [Class utf16be_encoding](#)
- [Class utf16le_encoding](#)
- [Class utf16bom_encoding](#)
- [Class utf32_encoding](#)
- [Class utf32be_encoding](#)
- [Class utf32le_encoding](#)
- [Class utf32bom_encoding](#)
- [Encoding type aliases](#)

Class trivial_encoding_state

```
class trivial_encoding_state {};
```

Class trivial_encoding_state_transition

```
class trivial_encoding_state_transition {};
```

Class basic_execution_character_encoding

```
class basic_execution_character_encoding {
public:
    using state_type = trivial_encoding_state;
    using state_transition_type = trivial_encoding_state_transition;
    using character_type = character<basic_execution_character_set>;
    using code_unit_type = char;

    static constexpr int min_code_units = 1;
    static constexpr int max_code_units = 1;

    static const state_type& initial_state();

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static void encode_state_transition(state_type &state,
                                           CUIT &out,
                                           const state_transition_type &stt,
                                           int &encoded_code_units)

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static void encode(state_type &state,
                           CUIT &out,
                           character_type c,
                           int &encoded_code_units)

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::InputIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static bool decode(state_type &state,
                           CUIT &in_next,
                           CUST in_end,
                           character_type &c,
                           int &decoded_code_units)

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::InputIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static bool rdecode(state_type &state,
                             CUIT &in_next,
                             CUST in_end,
                             character_type &c,
                             int &decoded_code_units)

};
```

Class basic_execution_wide_character_encoding

```

class basic_execution_wide_character_encoding {
public:
    using state_type = trivial_encoding_state;
    using state_transition_type = trivial_encoding_state_transition;
    using character_type = character<basic_execution_wide_character_set>;
    using code_unit_type = wchar_t;

    static constexpr int min_code_units = 1;
    static constexpr int max_code_units = 1;

    static const state_type& initial_state();

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static void encode_state_transition(state_type &state,
                                           CUIT &out,
                                           const state_transition_type &stt,
                                           int &encoded_code_units)

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static void encode(state_type &state,
                           CUIT &out,
                           character_type c,
                           int &encoded_code_units)

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::InputIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static bool decode(state_type &state,
                           CUIT &in_next,
                           CUST in_end,
                           character_type &c,
                           int &decoded_code_units)

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::InputIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static bool rdecode(state_type &state,
                             CUIT &in_next,
                             CUST in_end,
                             character_type &c,
                             int &decoded_code_units)
};

```

Class iso_10646_wide_character_encoding

```

#if defined(__STDC_ISO_10646__)
class iso_10646_wide_character_encoding {
public:
    using state_type = trivial_encoding_state;
    using state_transition_type = trivial_encoding_state_transition;
    using character_type = character<unicode_character_set>;
    using code_unit_type = wchar_t;

    static constexpr int min_code_units = 1;
    static constexpr int max_code_units = 1;

    static const state_type& initial_state();

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static void encode_state_transition(state_type &state,
                                           CUIT &out,
                                           const state_transition_type &stt,
                                           int &encoded_code_units)

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static void encode(state_type &state,
                           CUIT &out,
                           character_type c,
                           int &encoded_code_units)

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::InputIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static bool decode(state_type &state,

```



```

        CUIT &in_next,
        CUST in_end,
        character_type &c,
        int &decoded_code_units)

```

```

template<CodeUnitIterator CUIT, typename CUST>
    requires ranges::InputIterator<CUIT>()
        && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
        && ranges::Sentinel<CUST, CUIT>()
    static bool rdecode(state_type &state,
        CUIT &in_next,
        CUST in_end,
        character_type &c,
        int &decoded_code_units)

};

#endif // __STDC_ISO_10646__

```

Class utf8_encoding

```

class utf8_encoding {
public:
    using state_type = trivial_encoding_state;
    using state_transition_type = trivial_encoding_state_transition;
    using character_type = character<unicode_character_set>;
    using code_unit_type = char;

    static constexpr int min_code_units = 1;
    static constexpr int max_code_units = 4;

    static const state_type& initial_state();

    template<CodeUnitOutputIterator<std::make_unsigned_t<code_unit_type>> CUIT>
        static void encode_state_transition(state_type &state,
            CUIT &out,
            const state_transition_type &stt,
            int &encoded_code_units)

    template<CodeUnitOutputIterator<std::make_unsigned_t<code_unit_type>> CUIT>
        static void encode(state_type &state,
            CUIT &out,
            character_type c,
            int &encoded_code_units)

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::InputIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static bool decode(state_type &state,
            CUIT &in_next,
            CUST in_end,
            character_type &c,
            int &decoded_code_units)

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::InputIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static bool rdecode(state_type &state,
            CUIT &in_next,
            CUST in_end,
            character_type &c,
            int &decoded_code_units)

};

```

Class utf8bom_encoding

```

class utf8bom_encoding_state {
    /* implementation-defined */
};

class utf8bom_encoding_state_transition {
public:
    static utf8bom_encoding_state_transition to_initial_state();
    static utf8bom_encoding_state_transition to_bom_written_state();
    static utf8bom_encoding_state_transition to_assume_bom_written_state();
};

```

```

class utf8bom_encoding {
public:
    using state_type = utf8bom_encoding_state;
    using state_transition_type = utf8bom_encoding_state_transition;
    using character_type = character<unicode_character_set>;
    using code_unit_type = char;

    static constexpr int min_code_units = 1;
    static constexpr int max_code_units = 4;

    static const state_type& initial_state();

    template<CodeUnitOutputIterator<std::make_unsigned_t<code_unit_type>> CUIT>
        static void encode_state_transition(state_type &state,
                                           CUIT &out,
                                           const state_transition_type &stt,
                                           int &encoded_code_units)

    template<CodeUnitOutputIterator<std::make_unsigned_t<code_unit_type>> CUIT>
        static void encode(state_type &state,
                           CUIT &out,
                           character_type c,
                           int &encoded_code_units)

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::InputIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static bool decode(state_type &state,
                          CUIT &in_next,
                          CUST in_end,
                          character_type &c,
                          int &decoded_code_units)

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::InputIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static bool rdecode(state_type &state,
                           CUIT &in_next,
                           CUST in_end,
                           character_type &c,
                           int &decoded_code_units)

};

```

Class utf16_encoding

```

class utf16_encoding {
public:
    using state_type = trivial_encoding_state;
    using state_transition_type = trivial_encoding_state_transition;
    using character_type = character<unicode_character_set>;
    using code_unit_type = char16_t;

    static constexpr int min_code_units = 1;
    static constexpr int max_code_units = 2;

    static const state_type& initial_state();

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static void encode_state_transition(state_type &state,
                                           CUIT &out,
                                           const state_transition_type &stt,
                                           int &encoded_code_units)

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static void encode(state_type &state,
                           CUIT &out,
                           character_type c,
                           int &encoded_code_units)

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::InputIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static bool decode(state_type &state,

```

```

        CUIT &in_next,
        CUST in_end,
        character_type &c,
        int &decoded_code_units)

```

```

template<CodeUnitIterator CUIT, typename CUST>
    requires ranges::InputIterator<CUIT>()
        && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
        && ranges::Sentinel<CUST, CUIT>()
    static bool rdecode(state_type &state,
        CUIT &in_next,
        CUST in_end,
        character_type &c,
        int &decoded_code_units)
};

```

Class utf16be_encoding

```

class utf16be_encoding {
public:
    using state_type = trivial_encoding_state;
    using state_transition_type = trivial_encoding_state_transition;
    using character_type = character<unicode_character_set>;
    using code_unit_type = char;

    static constexpr int min_code_units = 2;
    static constexpr int max_code_units = 4;

    static const state_type& initial_state();

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static void encode_state_transition(state_type &state,
            CUIT &out,
            const state_transition_type &stt,
            int &encoded_code_units)

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static void encode(state_type &state,
            CUIT &out,
            character_type c,
            int &encoded_code_units)

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::InputIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static bool decode(state_type &state,
            CUIT &in_next,
            CUST in_end,
            character_type &c,
            int &decoded_code_units)

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::InputIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static bool rdecode(state_type &state,
            CUIT &in_next,
            CUST in_end,
            character_type &c,
            int &decoded_code_units)
};

```

Class utf16le_encoding

```

class utf16le_encoding {
public:
    using state_type = trivial_encoding_state;
    using state_transition_type = trivial_encoding_state_transition;
    using character_type = character<unicode_character_set>;
    using code_unit_type = char;

    static constexpr int min_code_units = 2;
    static constexpr int max_code_units = 4;

    static const state_type& initial_state();

```

```

template<CodeUnitOutputIterator<code_unit_type> CUIT>
    static void encode_state_transition(state_type &state,
                                       CUIT &out,
                                       const state_transition_type &stt,
                                       int &encoded_code_units)

template<CodeUnitOutputIterator<code_unit_type> CUIT>
    static void encode(state_type &state,
                      CUIT &out,
                      character_type c,
                      int &encoded_code_units)

template<CodeUnitIterator CUIT, typename CUST>
    requires ranges::InputIterator<CUIT>()
    && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
    && ranges::Sentinel<CUST, CUIT>()
    static bool decode(state_type &state,
                      CUIT &in_next,
                      CUST in_end,
                      character_type &c,
                      int &decoded_code_units)

template<CodeUnitIterator CUIT, typename CUST>
    requires ranges::InputIterator<CUIT>()
    && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
    && ranges::Sentinel<CUST, CUIT>()
    static bool rdecode(state_type &state,
                      CUIT &in_next,
                      CUST in_end,
                      character_type &c,
                      int &decoded_code_units)
};

```

Class utf16bom_encoding

```

class utf16bom_encoding_state {
    /* implementation-defined */
};

class utf16bom_encoding_state_transition {
public:
    static utf16bom_encoding_state_transition to_initial_state();
    static utf16bom_encoding_state_transition to_bom_written_state();
    static utf16bom_encoding_state_transition to_be_bom_written_state();
    static utf16bom_encoding_state_transition to_le_bom_written_state();
    static utf16bom_encoding_state_transition to_assume_bom_written_state();
    static utf16bom_encoding_state_transition to_assume_be_bom_written_state();
    static utf16bom_encoding_state_transition to_assume_le_bom_written_state();
};

class utf16bom_encoding {
public:
    using state_type = utf16bom_encoding_state;
    using state_transition_type = utf16bom_encoding_state_transition;
    using character_type = character<unicode_character_set>;
    using code_unit_type = char;

    static constexpr int min_code_units = 2;
    static constexpr int max_code_units = 4;

    static const state_type& initial_state();

template<CodeUnitOutputIterator<code_unit_type> CUIT>
    static void encode_state_transition(state_type &state,
                                       CUIT &out,
                                       const state_transition_type &stt,
                                       int &encoded_code_units)

template<CodeUnitOutputIterator<code_unit_type> CUIT>
    static void encode(state_type &state,
                      CUIT &out,
                      character_type c,
                      int &encoded_code_units)

template<CodeUnitIterator CUIT, typename CUST>
    requires ranges::InputIterator<CUIT>()

```

```

        && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
        && ranges::Sentinel<CUST, CUIT>()
    static bool decode(state_type &state,
                       CUIT &in_next,
                       CUST in_end,
                       character_type &c,
                       int &decoded_code_units)

template<CodeUnitIterator CUIT, typename CUST>
requires ranges::InputIterator<CUIT>()
    && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
    && ranges::Sentinel<CUST, CUIT>()
    static bool rdecode(state_type &state,
                        CUIT &in_next,
                        CUST in_end,
                        character_type &c,
                        int &decoded_code_units)
};

```

Class utf32_encoding

```

class utf32_encoding {
public:
    using state_type = trivial_encoding_state;
    using state_transition_type = trivial_encoding_state_transition;
    using character_type = character<unicode_character_set>;
    using code_unit_type = char32_t;

    static constexpr int min_code_units = 1;
    static constexpr int max_code_units = 1;

    static const state_type& initial_state();

template<CodeUnitOutputIterator<code_unit_type> CUIT>
    static void encode_state_transition(state_type &state,
                                       CUIT &out,
                                       const state_transition_type &stt,
                                       int &encoded_code_units)

template<CodeUnitOutputIterator<code_unit_type> CUIT>
    static void encode(state_type &state,
                       CUIT &out,
                       character_type c,
                       int &encoded_code_units)

template<CodeUnitIterator CUIT, typename CUST>
requires ranges::InputIterator<CUIT>()
    && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
    && ranges::Sentinel<CUST, CUIT>()
    static bool decode(state_type &state,
                       CUIT &in_next,
                       CUST in_end,
                       character_type &c,
                       int &decoded_code_units)

template<CodeUnitIterator CUIT, typename CUST>
requires ranges::InputIterator<CUIT>()
    && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
    && ranges::Sentinel<CUST, CUIT>()
    static bool rdecode(state_type &state,
                        CUIT &in_next,
                        CUST in_end,
                        character_type &c,
                        int &decoded_code_units)
};

```

Class utf32be_encoding

```

class utf32be_encoding {
public:
    using state_type = trivial_encoding_state;
    using state_transition_type = trivial_encoding_state_transition;
    using character_type = character<unicode_character_set>;
    using code_unit_type = char;

    static constexpr int min_code_units = 4;

```

```

static constexpr int max_code_units = 4;

static const state_type& initial_state();

template<CodeUnitOutputIterator<code_unit_type> CUIT>
    static void encode_state_transition(state_type &state,
                                       CUIT &out,
                                       const state_transition_type &stt,
                                       int &encoded_code_units)

template<CodeUnitOutputIterator<code_unit_type> CUIT>
    static void encode(state_type &state,
                      CUIT &out,
                      character_type c,
                      int &encoded_code_units)

template<CodeUnitIterator CUIT, typename CUST>
    requires ranges::InputIterator<CUIT>()
        && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
        && ranges::Sentinel<CUST, CUIT>()
    static bool decode(state_type &state,
                      CUIT &in_next,
                      CUST in_end,
                      character_type &c,
                      int &decoded_code_units)

template<CodeUnitIterator CUIT, typename CUST>
    requires ranges::InputIterator<CUIT>()
        && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
        && ranges::Sentinel<CUST, CUIT>()
    static bool rdecode(state_type &state,
                      CUIT &in_next,
                      CUST in_end,
                      character_type &c,
                      int &decoded_code_units)
};

```

Class utf32le_encoding

```

class utf32le_encoding {
public:
    using state_type = trivial_encoding_state;
    using state_transition_type = trivial_encoding_state_transition;
    using character_type = character<unicode_character_set>;
    using code_unit_type = char;

    static constexpr int min_code_units = 4;
    static constexpr int max_code_units = 4;

    static const state_type& initial_state();

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static void encode_state_transition(state_type &state,
                                           CUIT &out,
                                           const state_transition_type &stt,
                                           int &encoded_code_units)

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static void encode(state_type &state,
                          CUIT &out,
                          character_type c,
                          int &encoded_code_units)

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::InputIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static bool decode(state_type &state,
                          CUIT &in_next,
                          CUST in_end,
                          character_type &c,
                          int &decoded_code_units)

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::InputIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()

```

```

        static bool rdecode(state_type &state,
                            CUIT &in_next,
                            CUST in_end,
                            character_type &c,
                            int &decoded_code_units)

};

```

Class utf32bom_encoding

```

class utf32bom_encoding_state {
    /* implementation-defined */
};

class utf32bom_encoding_state_transition {
public:
    static utf32bom_encoding_state_transition to_initial_state();
    static utf32bom_encoding_state_transition to_bom_written_state();
    static utf32bom_encoding_state_transition to_be_bom_written_state();
    static utf32bom_encoding_state_transition to_le_bom_written_state();
    static utf32bom_encoding_state_transition to_assume_bom_written_state();
    static utf32bom_encoding_state_transition to_assume_be_bom_written_state();
    static utf32bom_encoding_state_transition to_assume_le_bom_written_state();
};

class utf32bom_encoding {
public:
    using state_type = utf32bom_encoding_state;
    using state_transition_type = utf32bom_encoding_state_transition;
    using character_type = character<unicode_character_set>;
    using code_unit_type = char;

    static constexpr int min_code_units = 4;
    static constexpr int max_code_units = 4;

    static const state_type& initial_state();

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static void encode_state_transition(state_type &state,
                                            CUIT &out,
                                            const state_transition_type &stt,
                                            int &encoded_code_units)

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static void encode(state_type &state,
                           CUIT &out,
                           character_type c,
                           int &encoded_code_units)

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::InputIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static bool decode(state_type &state,
                           CUIT &in_next,
                           CUST in_end,
                           character_type &c,
                           int &decoded_code_units)

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::InputIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static bool rdecode(state_type &state,
                             CUIT &in_next,
                             CUST in_end,
                             character_type &c,
                             int &decoded_code_units)

};

```

Encoding type aliases

```

using execution_character_encoding = /* implementation-defined */ ;
using execution_wide_character_encoding = /* implementation-defined */ ;
using char8_character_encoding = /* implementation-defined */ ;
using char16_character_encoding = /* implementation-defined */ ;
using char32_character_encoding = /* implementation-defined */ ;

```

Text Iterators

- [Class template itext_iterator](#)
- [Class template itext_sentinel](#)
- [Class template otext_iterator](#)
- [make_otext_iterator](#)

Class template itext_iterator

```
template<TextEncoding ET, ranges::InputRange RT>
    requires TextDecoder<
        ET,
        ranges::iterator_t<std::add_const_t<std::remove_reference_t<RT>>>>>()
class itext_iterator {
public:
    using encoding_type = ET;
    using range_type = std::remove_reference_t<RT>;
    using state_type = typename encoding_type::state_type;

    using iterator = ranges::iterator_t<std::add_const_t<range_type>>>;
    using iterator_category = /* implementation-defined */;
    using value_type = character_type_t<encoding_type>;
    using reference = std::add_const_t<value_type>&;
    using pointer = std::add_const_t<value_type>*;
    using difference_type = ranges::difference_type_t<iterator>;

    itext_iterator();

    itext_iterator(const state_type &state,
                  const range_type *range,
                  iterator first);

    reference operator*() const noexcept;
    pointer operator->() const noexcept;

    friend bool operator==(const itext_iterator &l, const itext_iterator &r);
    friend bool operator!=(const itext_iterator &l, const itext_iterator &r);

    friend bool operator<(const itext_iterator &l, const itext_iterator &r)
        requires TextRandomAccessDecoder<encoding_type, iterator>();
    friend bool operator>(const itext_iterator &l, const itext_iterator &r)
        requires TextRandomAccessDecoder<encoding_type, iterator>();
    friend bool operator<=(const itext_iterator &l, const itext_iterator &r)
        requires TextRandomAccessDecoder<encoding_type, iterator>();
    friend bool operator>=(const itext_iterator &l, const itext_iterator &r)
        requires TextRandomAccessDecoder<encoding_type, iterator>();

    itext_iterator& operator++();
    itext_iterator& operator++()
        requires TextForwardDecoder<encoding_type, iterator>();
    itext_iterator operator++(int);

    itext_iterator& operator--()
        requires TextBidirectionalDecoder<encoding_type, iterator>();
    itext_iterator operator--(int)
        requires TextBidirectionalDecoder<encoding_type, iterator>();

    itext_iterator& operator+=(difference_type n)
        requires TextRandomAccessDecoder<encoding_type, iterator>();
    itext_iterator& operator-=(difference_type n)
        requires TextRandomAccessDecoder<encoding_type, iterator>();

    friend itext_iterator operator+(itext_iterator l, difference_type n)
        requires TextRandomAccessDecoder<encoding_type, iterator>();
    friend itext_iterator operator+(difference_type n, itext_iterator r)
        requires TextRandomAccessDecoder<encoding_type, iterator>();

    friend itext_iterator operator-(itext_iterator l, difference_type n)
        requires TextRandomAccessDecoder<encoding_type, iterator>();
    friend difference_type operator-(const itext_iterator &l,
                                     const itext_iterator &r)
        requires TextRandomAccessDecoder<encoding_type, iterator>();

    value_type operator[](difference_type n) const
        requires TextRandomAccessDecoder<encoding_type, iterator>();
```



```

const state_type& state() const noexcept;
state_type& state() noexcept;

iterator base() const;

/* implementation-defined */ base_range() const
requires TextDecoder<encoding_type, iterator>()
    && ranges::ForwardIterator<iterator>();

bool is_ok() const noexcept;

private:
    state_type base_state; // exposition only
    iterator base_iterator; // exposition only
    bool ok; // exposition only
};

```

Class template itext_sentinel

```

template<TextEncoding ET, ranges::InputRange RT>
class itext_sentinel {
public:
    using range_type = std::remove_reference_t<RT>;
    using sentinel = ranges::sentinel_t<std::add_const_t<range_type>>>;

    itext_sentinel(sentinel s);

    itext_sentinel(const itext_iterator<ET, RT> &ti)
        requires ranges::ConvertibleTo<decltype(ti.base()), sentinel>();

    friend bool operator==(const itext_sentinel &l, const itext_sentinel &r);
    friend bool operator!=(const itext_sentinel &l, const itext_sentinel &r);

    friend bool operator==(const itext_iterator<ET, RT> &ti,
                           const itext_sentinel &ts);
    friend bool operator!=(const itext_iterator<ET, RT> &ti,
                           const itext_sentinel &ts);
    friend bool operator==(const itext_sentinel &ts,
                           const itext_iterator<ET, RT> &ti);
    friend bool operator!=(const itext_sentinel &ts,
                           const itext_iterator<ET, RT> &ti);

    friend bool operator<(const itext_sentinel &l, const itext_sentinel &r);
    friend bool operator>(const itext_sentinel &l, const itext_sentinel &r);
    friend bool operator<=(const itext_sentinel &l, const itext_sentinel &r);
    friend bool operator>=(const itext_sentinel &l, const itext_sentinel &r);

    friend bool operator<(const itext_iterator<ET, RT> &ti,
                           const itext_sentinel &ts)
        requires ranges::StrictWeakOrder<
            std::less<>,
            typename itext_iterator<ET, RT>::iterator,
            sentinel>();
    friend bool operator>(const itext_iterator<ET, RT> &ti,
                           const itext_sentinel &ts)
        requires ranges::StrictWeakOrder<
            std::less<>,
            typename itext_iterator<ET, RT>::iterator,
            sentinel>();
    friend bool operator<=(const itext_iterator<ET, RT> &ti,
                           const itext_sentinel &ts)
        requires ranges::StrictWeakOrder<
            std::less<>,
            typename itext_iterator<ET, RT>::iterator,
            sentinel>();
    friend bool operator>=(const itext_iterator<ET, RT> &ti,
                           const itext_sentinel &ts)
        requires ranges::StrictWeakOrder<
            std::less<>,
            typename itext_iterator<ET, RT>::iterator,
            sentinel>();

    friend bool operator<(const itext_sentinel &ts,
                           const itext_iterator<ET, RT> &ti)
        requires ranges::StrictWeakOrder<
            std::less<>,

```

```

        typename itext_iterator<ET, RT>::iterator,
        sentinel>();
friend bool operator>(const itext_sentinel &ts,
                    const itext_iterator<ET, RT> &ti)
    requires ranges::StrictWeakOrder<
        std::less<>,
        typename itext_iterator<ET, RT>::iterator,
        sentinel>();
friend bool operator<=(const itext_sentinel &ts,
                    const itext_iterator<ET, RT> &ti)
    requires ranges::StrictWeakOrder<
        std::less<>,
        typename itext_iterator<ET, RT>::iterator,
        sentinel>();
friend bool operator>=(const itext_sentinel &ts,
                    const itext_iterator<ET, RT> &ti)
    requires ranges::StrictWeakOrder<
        std::less<>,
        typename itext_iterator<ET, RT>::iterator,
        sentinel>();

    sentinel base() const;

private:
    sentinel base_sentinel; // exposition only
};

```

Class template otext_iterator

```

template<TextEncoding E, CodeUnitOutputIterator<code_unit_type_t<E>> CUIT>
class otext_iterator {
public:
    using encoding_type = E;
    using state_type = typename E::state_type;
    using state_transition_type = typename E::state_transition_type;

    using iterator = CUIT;
    using iterator_category = std::output_iterator_tag;
    using value_type = character_type_t<encoding_type>;
    using reference = value_type&;
    using pointer = value_type*;
    using difference_type = ranges::difference_type_t<iterator>;

    otext_iterator();

    otext_iterator(state_type state, iterator current);

    otext_iterator& operator*();

    otext_iterator& operator++();
    otext_iterator& operator++(int);

    otext_iterator& operator=(const state_transition_type &stt);
    otext_iterator& operator=(const character_type_t<encoding_type> &value);

    const state_type& state() const noexcept;
    state_type& state() noexcept;

    iterator base() const;

private:
    state_type base_state; // exposition only
    iterator base_iterator; // exposition only
};

```

make_otext_iterator

```

template<TextEncoding ET, CodeUnitOutputIterator<code_unit_type_t<ET>> IT>
    auto make_otext_iterator(typename ET::state_type state, IT out)
    -> otext_iterator<ET, IT>;
template<TextEncoding ET, CodeUnitOutputIterator<code_unit_type_t<ET>> IT>
    auto make_otext_iterator(IT out)
    -> otext_iterator<ET, IT>;

```

Text View

- [Class template basic_text_view](#)
- [Text view type aliases](#)
- [make_text_view](#)

Class template basic_text_view

```
template<TextEncoding ET, ranges::InputRange RT>
class basic_text_view {
public:
    using encoding_type = ET;
    using range_type = RT;
    using state_type = typename ET::state_type;
    using code_unit_iterator = ranges::iterator_t<std::add_const_t<range_type>>;
    using code_unit_sentinel = ranges::sentinel_t<std::add_const_t<range_type>>;
    using iterator = itext_iterator<ET, RT>;
    using sentinel = itext_sentinel<ET, RT>;

    basic_text_view();

    basic_text_view(state_type state,
                    range_type r)
        requires ranges::CopyConstructible<range_type>();

    basic_text_view(range_type r)
        requires ranges::CopyConstructible<range_type>();

    basic_text_view(state_type state,
                    code_unit_iterator first,
                    code_unit_sentinel last)
        requires ranges::Constructible<range_type,
                                     code_unit_iterator,
                                     code_unit_sentinel>();

    basic_text_view(code_unit_iterator first,
                    code_unit_sentinel last)
        requires ranges::Constructible<range_type,
                                     code_unit_iterator,
                                     code_unit_sentinel>();

    basic_text_view(state_type state,
                    code_unit_iterator first,
                    ranges::difference_type_t<code_unit_iterator> n)
        requires ranges::Constructible<range_type,
                                     code_unit_iterator,
                                     code_unit_iterator>();

    basic_text_view(code_unit_iterator first,
                    ranges::difference_type_t<code_unit_iterator> n)
        requires ranges::Constructible<range_type,
                                     code_unit_iterator,
                                     code_unit_iterator>();

    template<typename charT, typename traits, typename Allocator>
    basic_text_view(state_type state,
                    const basic_string<charT, traits, Allocator> &str)
        requires ranges::Constructible<code_unit_iterator, const charT *>()
        && ranges::Constructible<ranges::difference_type_t<code_unit_iterator>,
                                typename basic_string<charT, traits, Allocator>::size_type>()
        && ranges::Constructible<range_type,
                                code_unit_iterator,
                                code_unit_sentinel>();

    template<typename charT, typename traits, typename Allocator>
    basic_text_view(const basic_string<charT, traits, Allocator> &str)
        requires ranges::Constructible<code_unit_iterator, const charT *>()
        && ranges::Constructible<ranges::difference_type_t<code_unit_iterator>,
                                typename basic_string<charT, traits, Allocator>::size_type>()
        && ranges::Constructible<range_type,
                                code_unit_iterator,
                                code_unit_sentinel>();

    template<ranges::InputRange Iterable>
    basic_text_view(state_type state,
                    const Iterable &iterable)
        requires ranges::Constructible<code_unit_iterator,
                                     ranges::iterator_t<const Iterable>>()
```

```

        && ranges::Constructible<range_type,
                                code_unit_iterator,
                                code_unit_sentinel>());

template<ranges::InputRange Iterable>
    basic_text_view(const Iterable &iterable)
    requires ranges::Constructible<code_unit_iterator,
                                    ranges::iterator_t<const Iterable>>()
        && ranges::Constructible<range_type,
                                code_unit_iterator,
                                code_unit_sentinel>());

    basic_text_view(iterator first, sentinel last)
    requires ranges::Constructible<code_unit_iterator,
                                    decltype(std::declval<iterator>().base())>()
        && ranges::Constructible<range_type,
                                code_unit_iterator,
                                code_unit_sentinel>());

    const range_type& base() const noexcept;
    range_type& base() noexcept;

    const state_type& initial_state() const noexcept;
    state_type& initial_state() noexcept;

    iterator begin() const;
    iterator end() const
        requires std::is_empty<state_type>::value
        && ranges::Iterator<code_unit_sentinel>();
    sentinel end() const
        requires !std::is_empty<state_type>::value
        || !ranges::Iterator<code_unit_sentinel>();

private:
    state_type base_state; // exposition only
    range_type base_range; // exposition only
};

```

Text view type aliases

```

using text_view = basic_text_view<
    execution_character_encoding,
    /* implementation-defined */ >;
using wtext_view = basic_text_view<
    execution_wide_character_encoding,
    /* implementation-defined */ >;
using u8text_view = basic_text_view<
    char8_character_encoding,
    /* implementation-defined */ >;
using u16text_view = basic_text_view<
    char16_character_encoding,
    /* implementation-defined */ >;
using u32text_view = basic_text_view<
    char32_character_encoding,
    /* implementation-defined */ >;

```

make_text_view

```

template<TextEncoding ET, ranges::InputIterator IT, ranges::Sentinel<IT> ST>
    auto make_text_view(typename ET::state_type state,
                        IT first, ST last)
    -> basic_text_view<ET, /* implementation-defined */ >;

template<TextEncoding ET, ranges::InputIterator IT, ranges::Sentinel<IT> ST>
    auto make_text_view(IT first, ST last)
    -> basic_text_view<ET, /* implementation-defined */ >;

template<TextEncoding ET, ranges::ForwardIterator IT>
    auto make_text_view(typename ET::state_type state,
                        IT first,
                        ranges::difference_type_t<IT> n)
    -> basic_text_view<ET, /* implementation-defined */ >;

template<TextEncoding ET, ranges::ForwardIterator IT>
    auto make_text_view(IT first,

```

```

        ranges::difference_type_t<IT> n)
-> basic_text_view<ET, /* implementation-defined */ >;

template<TextEncoding ET, ranges::InputRange Iterable>
    auto make_text_view(typename ET::state_type state,
        const Iterable &iterable)
-> basic_text_view<ET, /* implementation-defined */ >;

template<TextEncoding ET, ranges::InputRange Iterable>
    auto make_text_view(const Iterable &iterable)
-> basic_text_view<ET, /* implementation-defined */ >;

template<TextIterator TIT, TextSentinel<TIT> TST>
    auto make_text_view(TIT first, TST last)
-> basic_text_view<ET, /* implementation-defined */ >;

template<TextView TVT>
    TVT make_text_view(TVT tv);

```

Acknowledgements

Thank you to the std-proposals community and especially to Zhihao Yuan, Jeffrey Yasskin, Thiago Macieira, and Nicol Bolas for their design feedback.

References

- [C++11] "Information technology -- Programming languages -- C++", ISO/IEC 14882:2011.
http://www.iso.org/iso/home/store/catalogue_ics/catalogue_detail_ics.htm?csnumber=50372
- [Concepts] "C++ Extensions for concepts", ISO/IEC technical specification 19217:2015.
http://www.iso.org/iso/home/store/catalogue_tc/catalogue_detail.htm?csnumber=64031
- [N2249] Lawrence Crowl, "New Character Types in C++", N2249, 2007.
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2249.html>
- [N2442] Lawrence Crowl and Beman Dawes, "Raw and Unicode String Literals; Unified Proposal (Rev. 2)", N2442, 2007.
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2442.htm>
- [Origin] Andrew Sutton, Origin libraries.
<http://asutton.github.io/origin>
- [Proxy Iterators] Eric Niebler, "Proxy Iterators for the Ranges Extensions", P0022R1, 2015.
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/p0022r1.html>
- [Ranges] Eric Niebler and Casey Carter, "Working Draft, C++ Extensions for Ranges", N4560, 2015.
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4560.pdf>
- [Text_view] Tom Honermann, Text_view library.
https://github.com/tahonermann/text_view
- [Unicode] "Unicode 8.0.0", 2015.
<http://www.unicode.org/versions/Unicode8.0.0>